实验 1 - PyTorch 基础与线性模型

数据科学与大数据技术专业

第 1–3 周

1 实验目标

- 掌握 PyTorch 张量 (`torch.Tensor`) 的基本操作及自动微分 (`autograd`) 机制.
- 理解并实现线性回归模型, 掌握经验风险最小化 (ERM) 与结构风险最小化 (SRM / L2 正则化) 的区别.
- 掌握线性分类模型的 PyTorch 实现, 包括 Logistic 回归、感知器与支持向量机 (SVM).
- 实现多分类线性模型 (Softmax 回归), 并在 MNIST 数据集上训练与评估.
- 在不借助任何高级 API 的前提下, 从零手写 Logistic 回归的前向传播、损失函数与梯度下降.
- 掌握 PyTorch 基础操作 (线性函数、Sigmoid、One-Hot 编码、Xavier 初始化), 并搭建一个三层全连接神经网络在手势数字数据集上进行多分类训练.
- 熟悉 `DataLoader` / `TensorDataset` 的使用, 理解小批量梯度下降 (Mini-batch SGD) 与 Adam 优化器的作用.

2 实验环境

- 操作系统: Windows / Linux / macOS
- Python 3.7 及以上
- 主要依赖库: `torch`、`torchvision`、`numpy`、`matplotlib`、`seaborn`、`h5py`
- 推荐开发环境: Jupyter Notebook 或 VS Code

3 背景知识

3.1 机器学习四要素与风险函数

机器学习的核心要素包括数据、模型、学习准则与优化算法. 给定训练集 $\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$, 模型参数 $\boldsymbol{w}$ 的学习通常通过最小化某种风险函数来完成.

期望风险 (Expected Risk):

$$R(\boldsymbol{w}) = \mathbb{E}\left[\mathcal{L}\left(y, f(\boldsymbol{x}; \boldsymbol{w})\right)\right] \tag{3.1}$$

由于真实数据分布未知, 实际中以**经验风险 (Empirical Risk)** 来近似:

$$\hat{R}_D(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{L}\left(y^{(n)}, f(\boldsymbol{x}^{(n)};\boldsymbol{w})\right) \tag{3.2}$$

经验风险最小化 (ERM) 直接最小化 $\hat{R}_D(\boldsymbol{w})$, 但可能导致过拟合.

结构风险最小化 (SRM) 在经验风险的基础上引入正则化惩罚项 (如 L2 正则化):

$$\mathcal{R}_{srm}(\boldsymbol{w}) = \hat{R}_D(\boldsymbol{w}) + \frac{\lambda}{2}\|\boldsymbol{w}\|^2 \tag{3.3}$$

其中 $\lambda > 0$ 为正则化超参数, 用于控制模型复杂度, 缓解过拟合.

3.2 线性回归

线性回归模型通过特征的加权组合来预测连续输出. 采用均方误差 (MSE) 作为损失函数时, 经验风险为:

$$\mathcal{R}(\boldsymbol{w}) = \frac{1}{2}\left\|\boldsymbol{y} - \boldsymbol{X}^\top\boldsymbol{w}\right\|^2 \tag{3.4}$$

其中 $\boldsymbol{X} = [\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(N)}]$ 为特征矩阵, $\boldsymbol{y} = [y^{(1)}, \ldots, y^{(N)}]^\top$ 为标签向量.

ERM 的解析最优解 (正规方程) 为:

$$\boldsymbol{w}^*_{ERM} = \left(\boldsymbol{X}\boldsymbol{X}^\top\right)^{-1}\boldsymbol{X}\boldsymbol{y} \tag{3.5}$$

引入 L2 正则化后 (SRM / 岭回归) 的解析最优解为:

$$\boldsymbol{w}^*_{SRM} = \left(\boldsymbol{X}\boldsymbol{X}^\top + \lambda\boldsymbol{I}\right)^{-1}\boldsymbol{X}\boldsymbol{y} \tag{3.6}$$

在 PyTorch 中, L2 正则化可通过优化器的 `weight_decay` 参数直接实现.

3.3 前向传播与反向传播

前向传播 (Forward Propagation) 指将输入 $\boldsymbol{x}$ 经过网络各层运算得到预测输出 $\hat{y}$ 的过程. 以线性回归为例:

$$\hat{y} = \boldsymbol{w}^\top\boldsymbol{x} + b \tag{3.7}$$

反向传播 (Backward Propagation) 基于链式法则 (Chain Rule) 计算损失函数 $\mathcal{L}$ 对各参数的梯度:

$$\frac{\partial\mathcal{L}}{\partial\boldsymbol{w}} = \frac{\partial\mathcal{L}}{\partial\hat{y}}\cdot\frac{\partial\hat{y}}{\partial\boldsymbol{w}} \tag{3.8}$$

在 PyTorch 中, 调用 `loss.backward()` 即可自动完成反向传播, 随后调用 `optimizer.step()` 更新参数.

3.4 Logistic 回归 (二分类)

Logistic 回归利用 Sigmoid 函数将线性输出映射为概率:

$$\hat{y} = \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}\right) = \frac{1}{1+\exp\left(-\boldsymbol{w}^\top \boldsymbol{x}\right)} \tag{3.9}$$

采用二元交叉熵 (Binary Cross-Entropy, BCE) 作为损失函数:

$$\mathcal{R}(\boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N}\left[y^{(n)}\log\hat{y}^{(n)} + \left(1-y^{(n)}\right)\log\left(1-\hat{y}^{(n)}\right)\right] \tag{3.10}$$

该损失关于参数 $\boldsymbol{w}$ 的梯度 (推导结果极为简洁) 为:

$$\frac{\partial\mathcal{R}(\boldsymbol{w})}{\partial\boldsymbol{w}} = \frac{1}{N}\sum_{n=1}^{N}\left(\hat{y}^{(n)} - y^{(n)}\right)\boldsymbol{x}^{(n)} \tag{3.11}$$

参数更新规则 (梯度下降, 学习率为 α):

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t - \alpha\cdot\frac{\partial\mathcal{R}(\boldsymbol{w}_t)}{\partial\boldsymbol{w}_t} \tag{3.12}$$

在 PyTorch 中, `BCEWithLogitsLoss` 将 Sigmoid 激活与 BCE 损失合并, 数值上更为稳定.

3.5 感知器 (Perceptron)

感知器是最早的线性分类模型, 采用符号函数进行决策:

$$\hat{y} = \operatorname{sign}\left(\boldsymbol{w}^\top \boldsymbol{x}\right) \tag{3.13}$$

标签 $y \in \{+1, -1\}$. 感知器对误分类样本 (即满足 $y\left(\boldsymbol{w}^\top \boldsymbol{x}\right) \le 0$ 的样本) 进行参数更新:

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + y\boldsymbol{x} \tag{3.14}$$

其损失函数形式为 $\mathcal{L} = \max(0, -y(\boldsymbol{w}^\top \boldsymbol{x}))$. 前提条件: 数据需线性可分, 否则算法不收敛.

3.6 支持向量机 (SVM)

SVM 的核心思想是寻找最大间隔超平面. 软间隔 SVM 的目标函数为:

$$\min_{\boldsymbol{w},b}\sum_{n=1}^{N}\max\left(0,\, 1-y^{(n)}\left(\boldsymbol{w}^\top \boldsymbol{x}^{(n)}+b\right)\right) + \frac{1}{2C}\|\boldsymbol{w}\|^2 \tag{3.15}$$

其中前半部分为 Hinge 损失, 后半部分为正则化项, 超参数 C 用于调节两者平衡.

Hinge 损失的定义:

$$\mathcal{L}_{hinge} = \max\left(0,\, 1-y\cdot f(\boldsymbol{x})\right) \tag{3.16}$$

在 PyTorch 中可通过 `torch.clamp(1 - y * y_pred, min=0)` 计算, 并结合 `weight_decay` 实现正则化.

3.7 Softmax 回归 (多分类)

Softmax 回归是 Logistic 回归在多分类问题上的推广. 对于 C 个类别, Softmax 函数将线性输出的实数得分 $\boldsymbol{z} \in \mathbb{R}^C$ 映射为概率分布:

$$\mathrm{softmax}(z_c) = \frac{\exp(z_c)}{\sum_{i=1}^{C} \exp(z_i)}, \quad c = 1, \ldots, C \tag{3.17}$$

对应的多元交叉熵损失 (Categorical Cross-Entropy) 为:

$$\mathcal{L} = -\sum_{c=1}^{C} \mathbb{I}[y = c] \log \mathrm{softmax}(z_c) \tag{3.18}$$

在 PyTorch 中, `CrossEntropyLoss` 将 Softmax 与交叉熵合并 (输入为未归一化的 logits, 标签为类别索引), 数值更加稳定.

3.8 信息论基础: 交叉熵与 KL 散度

信息熵 (Entropy) 衡量分布 P 的不确定性:

$$H(P) = -\sum_{x} P(x) \log P(x) \tag{3.19}$$

交叉熵 (Cross-Entropy) 衡量用预测分布 Q 编码真实分布 P 所需的平均信息量:

$$H(P, Q) = -\sum_{x} P(x) \log Q(x) \tag{3.20}$$

KL 散度 (Kullback-Leibler Divergence), 又称相对熵, 量化两个分布之间的差异:

$$D_{\mathrm{KL}}(P \| Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)} = H(P, Q) - H(P) \tag{3.21}$$

最小化交叉熵损失等价于最小化 $D_{\mathrm{KL}}(P \| Q)$(因为 $H(P)$ 与模型参数无关), 也等价于对真实数据分布做极大似然估计 (MLE).

3.9 Xavier 初始化与 One-Hot 编码

Xavier / Glorot 初始化: 权重从均值为 0、方差为 $\frac{2}{n_{in}+n_{out}}$ 的分布中采样, 有助于保持各层激活值的方差一致, 缓解梯度消失/爆炸:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right) \tag{3.22}$$

在 PyTorch 中使用 `nn.init.xavier_normal_(param)` 实现.

One-Hot 编码将整数类别标签 $y \in \{0, \ldots, C-1\}$ 转化为长度为 C 的 0/1 向量, 便于可视化与理解. 在 PyTorch 中使用 `F.one_hot(label, num_classes=C)` 实现.

3.10 PyTorch `DataLoader` 与训练循环

`TensorDataset` 将特征张量与标签张量封装为数据集, `DataLoader` 在其上进行分批 (mini-batch) 和随机打乱 (shuffle), 支持高效的迭代训练. 标准 PyTorch 训练循环如下:

1. `optimizer.zero_grad()`: 清零上一步累积的梯度.
2. 前向传播: 调用模型或手写前向函数, 得到预测输出.
3. 计算损失: 将预测输出与真实标签代入损失函数.
4. `loss.backward()`: 反向传播, 自动计算所有参数的梯度.
5. `optimizer.step()`: 按照优化规则更新参数.

Adam 优化器结合了 Momentum 与 RMSProp, 自适应调节各参数的学习率:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \tag{3.23}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{3.24}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{3.25}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{3.26}$$

默认超参数: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

4 实验步骤

实验 1: PyTorch 基础与线性模型

1.1 线性回归

实验目标: 实现并比较 ERM(纯均方误差) 与 SRM(带 L2 正则化的均方误差) 下的线性回归模型.

定义与公式:

- **数据生成**: $y = 2x + 3 + \varepsilon$, 其中 $\varepsilon \sim \mathcal{N}(0, 4)$.
- **ERM 损失 (MSE)**:

$$\mathcal{L}_{ERM} = \frac{1}{N} \sum_{n=1}^{N} \left(\hat{y}^{(n)} - y^{(n)} \right)^2 \tag{4.1}$$

- **SRM 损失 (MSE + L2 正则化)**:

$$\mathcal{L}_{SRM} = \frac{1}{N} \sum_{n=1}^{N} \left(\hat{y}^{(n)} - y^{(n)} \right)^2 + \lambda \|\boldsymbol{w}\|^2 \tag{4.2}$$

PyTorch 关键 API:

- 模型定义: `nn.Linear(in_features, out_features)`

- 损失函数: `nn.MSELoss()`
- 优化器: `torch.optim.SGD(model.parameters(), lr=0.01)`
- L2 正则化: `torch.optim.SGD(..., weight_decay=λ)`

实验细节:

- 分别使用 ERM 和 SRM(`weight_decay=1.5`) 训练同一线性模型 100 个 epoch.
- 绘制损失曲线, 对比两种设置下的收敛速度与最终参数值.
- 观察较大的 `weight_decay` 如何将学习到的权重向零压缩.

1.2 线性分类

实验目标: 在二维二分类数据集上分别实现 Logistic 回归、感知器和 SVM, 并可视化各模型的决策边界.

(a) Logistic 回归　公式:

$$\hat{y} = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b) = \frac{1}{1 + e^{-(\boldsymbol{w}^\top \boldsymbol{x} + b)}} \tag{4.3}$$

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^{N} \left[y^{(n)} \log \sigma(z^{(n)}) + (1 - y^{(n)}) \log(1 - \sigma(z^{(n)})) \right] \tag{4.4}$$

PyTorch 关键 API: `nn.BCEWithLogitsLoss()`(数值上等价于先过 Sigmoid 再计算 BCE, 但更稳定).

(b) 感知器　算法 (误分类驱动更新): 标签 $y \in \{+1, -1\}$, 对每个样本判断 $y^{(n)}(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b) \leq 0$ 时执行更新:

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\, y^{(n)} \boldsymbol{x}^{(n)}, \quad b \leftarrow b + \eta\, y^{(n)} \tag{4.5}$$

此部分直接使用 PyTorch 张量操作手动实现, 不借助 `torch.optim`.

(c) 线性 SVM(Hinge Loss)　公式:

$$\mathcal{L}_{hinge} = \frac{1}{N} \sum_{n=1}^{N} \max\left(0,\, 1 - y^{(n)}(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b)\right) + \frac{\lambda}{2} \|\boldsymbol{w}\|^2 \tag{4.6}$$

PyTorch 实现: 使用 `torch.clamp(1 - y_perc * y_pred, min=0).mean()` 计算 Hinge 损失, 结合 `weight_decay` 实现正则化项.

1.3 多分类线性模型 (Softmax 回归)

实验目标: 在 MNIST 数据集 (10 类手写数字) 上训练单层 Softmax 线性分类器.
模型架构: 输入为 $28 \times 28 = 784$ 维展平图像向量, 输出为 10 维 logits(未归一化分数):

$$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}, \quad \boldsymbol{W} \in \mathbb{R}^{10 \times 784} \tag{4.7}$$

损失函数: 多元交叉熵, PyTorch 提供 `nn.CrossEntropyLoss()`(内部自动处理 Softmax).

实验细节:

- 使用 `torchvision.datasets.MNIST` 加载数据, `DataLoader` 分批 (batch size=64).
- 图像形状为 `(batch_size, 1, 28, 28)`, 需展平为 `(batch_size, 784)`, 即 `X.view(-1, 28*28)`.
- 训练 10 个 epoch, 打印每 epoch 的训练与测试损失.
- 绘制混淆矩阵 (Confusion Matrix), 分析各数字的分类情况.

实验 2: 纯 PyTorch 手写 Logistic 回归

实验目标: 在不使用任何高级 API(如 `nn.Linear`、`nn.BCELoss`、`torch.optim`) 的前提下, 完全基于 PyTorch 张量操作手动实现 Logistic 回归, 深入理解神经网络训练的底层机制.

手动实现要点:

- **参数初始化**: 直接使用 `requires_grad=True` 创建权重 $\boldsymbol{W} \in \mathbb{R}^{2\times 1}$ 与偏置 $b \in \mathbb{R}^{1\times 1}$.
- **前向传播**:

$$\boldsymbol{z} = \boldsymbol{X}\boldsymbol{W} + b \quad (\text{形状: } (m, 1)) \tag{4.8}$$

$$\hat{\boldsymbol{y}} = \sigma(\boldsymbol{z}) = \frac{1}{1 + e^{-\boldsymbol{z}}} \tag{4.9}$$

- **损失函数 (手动 BCE)**:

$$\mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\hat{y}^{(i)} + (1 - y^{(i)})\log(1 - \hat{y}^{(i)})\right] \tag{4.10}$$

  为防止 $\log(0)$ 导致数值不稳定, 需将 $\hat{y}$ 截断至 $[10^{-7}, 1 - 10^{-7}]$(使用 `torch.clamp`).

- **反向传播**: 调用 `loss.backward()` 自动计算 $\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}}$ 与 $\frac{\partial \mathcal{L}}{\partial b}$.
- **手动参数更新** (在 `torch.no_grad()` 上下文中执行, 以避免梯度追踪):

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \cdot \nabla_{\boldsymbol{W}}\mathcal{L} \tag{4.11}$$

$$b \leftarrow b - \eta \cdot \nabla_{b}\mathcal{L} \tag{4.12}$$

- **梯度清零**: 每步更新后调用 `W.grad.zero_()`、`b.grad.zero_()`.

实验细节:

- 数据: 两类高斯分布的二维数据, 类别 0 均值 $(-2, -2)$, 类别 1 均值 $(2, 2)$, 各 100 个样本.
- 训练 200 个 epoch, 学习率 $\eta = 0.1$.
- 绘制训练损失曲线与最终决策边界 ($P(y = 1|\boldsymbol{x}) = 0.5$ 的等值线).
- 计算并输出训练集准确率.

W3A1 PyTorch: 基础操作与三层神经网络

实验目标: 掌握 PyTorch 的核心基础操作, 并搭建一个三层全连接神经网络, 在手势数字 (Hand Sign) 数据集上完成 6 分类任务.

基础操作练习

线性函数 实现 $\boldsymbol{Y} = \boldsymbol{W}\boldsymbol{X} + \boldsymbol{b}$, 其中 $\boldsymbol{W} \in \mathbb{R}^{4\times 3}$, $\boldsymbol{X} \in \mathbb{R}^{3\times 1}$, $\boldsymbol{b} \in \mathbb{R}^{4\times 1}$. 使用 `torch.tensor(..., dtype=torch.float32)` 创建张量, `torch.matmul` 进行矩阵乘法.

Sigmoid 函数 将标量或 Python 数值转换为 `float32` 张量, 调用 `torch.sigmoid(z)`.

One-Hot 编码 将整数标签 $\ell \in \{0, \ldots, C-1\}$ 转为长度为 C 的 one-hot 向量:

$$\boldsymbol{e}_\ell = [\underbrace{0, \ldots, 0}_{\ell}, 1, \underbrace{0, \ldots, 0}_{C-\ell-1}]^\top \in \mathbb{R}^C \tag{4.13}$$

使用 `F.one_hot(torch.as_tensor(label, dtype=torch.long), num_classes=C).float()`.

参数初始化 (Xavier / Glorot Normal) 初始化三层网络的权重与偏置:

- $\boldsymbol{W}_1 \in \mathbb{R}^{25\times 12288}$, $\boldsymbol{b}_1 \in \mathbb{R}^{25\times 1}$
- $\boldsymbol{W}_2 \in \mathbb{R}^{12\times 25}$, $\boldsymbol{b}_2 \in \mathbb{R}^{12\times 1}$
- $\boldsymbol{W}_3 \in \mathbb{R}^{6\times 12}$, $\boldsymbol{b}_3 \in \mathbb{R}^{6\times 1}$

权重使用 `nn.init.xavier_normal_(torch.empty(shape))` 初始化, 偏置初始化为零: `torch.zeros(shape)`.

三层神经网络

网络架构: $X \xrightarrow{\text{LINEAR}} Z_1 \xrightarrow{\text{ReLU}} A_1 \xrightarrow{\text{LINEAR}} Z_2 \xrightarrow{\text{ReLU}} A_2 \xrightarrow{\text{LINEAR}} Z_3$

前向传播公式:

$$Z_1 = \boldsymbol{W}_1 X + \boldsymbol{b}_1 \tag{4.14}$$

$$A_1 = \mathrm{ReLU}(Z_1) = \max(0, Z_1) \tag{4.15}$$

$$Z_2 = \boldsymbol{W}_2 A_1 + \boldsymbol{b}_2 \tag{4.16}$$

$$A_2 = \mathrm{ReLU}(Z_2) \tag{4.17}$$

$$Z_3 = \boldsymbol{W}_3 A_2 + \boldsymbol{b}_3 \tag{4.18}$$

其中输入 X 的形状为 $(12288, N)$(特征维度 $\times$ 批大小), Z_3 为 $(6, N)$ 的 logits 输出.

损失函数 (多元交叉熵):

$$\mathcal{L}_{total} = \sum_{i=1}^{N} \mathcal{L}_{CE}\left(Z_3^{(i)}, y^{(i)}\right) \tag{4.19}$$

使用 `F.cross_entropy(Z3.T, labels, reduction='sum')` 实现 (注意需要转置 Z_3 至形状 $(N, 6)$).

训练设置:

- 数据集: Hand Sign 手势数字数据集, 训练集 1080 张、测试集 120 张, 每张图像为 $64 \times 64 \times 3$, 展平后维度为 12288.
- 预处理: 像素值归一化至 $[0, 1]$, 即除以 255.
- 优化器: Adam, 学习率 $\eta = 0.0001$.
- 训练 1000 个 epoch, 小批量大小为 32.
- 使用 `DataLoader(TensorDataset(X_train, Y_train), batch_size=32, shuffle=True)`.

5 各线性模型对比总结

模型	激活函数	损失函数	优化方法	应用场景
线性回归 (ERM)	无	均方误差 (MSE)	SGD / 最小二乘	连续值回归
线性回归 (SRM)	无	MSE + L2 正则化	SGD + `weight_decay`	回归 (防过拟合)
Logistic 回归	Sigmoid	二元交叉熵 (BCE)	SGD / Adam	二分类概率预测
Softmax 回归	Softmax	多元交叉熵 (CE)	SGD / Adam	多分类概率预测
感知器	sign	错误驱动 (Hinge 样)	在线 SGD	线性可分二分类
SVM(软间隔)	sign	Hinge + L2 正则化	SGD / QP / SMO	最大间隔二分类

6 实验作业

对于学有余力且富有创造力的同学, 除上述基础实验外, 还可以完成以下补充选做内容, 用于探索更完整的工程实践与更开放的创意应用. *请务必仔细阅读最后一节关于作业的重要说明.*

1. 按照实验 1 中的提示补全三个实验 (线性回归、线性分类、Softmax 回归) 的代码, 并观察各模型的损失曲线和准确率.
2. 按照实验 2 的提示, 从零实现 Logistic 回归的前向传播、BCE 损失、反向传播和参数更新, 观察训练损失下降趋势及决策边界.
3. 按照实验 3 的提示, 完成编程练习.
4. **选做**: 在实验 1 中, 尝试不同的 `weight_decay` 值 (如 $0, 0.1, 1.0, 5.0$), 分析正则化强度对模型权重和损失的影响.
5. **选做**: 在实验 2 中, 将手动梯度下降替换为使用 `torch.optim.SGD` 的版本, 验证两者结果是否一致.
6. **选做**: 补充和完善课程知识文档或环境配置说明. 例如, 在现有 Mac 配置说明的基础上, 增加 Windows 环境下的安装与运行步骤, 或补充校园网、代理网络、镜像源等特殊网络环境中的配置方法; 鼓励将整理后的 Markdown 文档提交到课程代码仓库, 共同完善课程资料.
7. **选做**: 基于 PyTorch 与线性模型实现一个有意思的小项目或创意应用. 你可以尝试用线性回归、Logistic 回归、Softmax 回归、感知器或 SVM 解决一个自选问题, 也可以在基础框架上探索更有想象力的应用形式; 提交时需同时说明你的任务设定、实现思路、实验结果以及创意亮点.

7 关于作业的重要说明

现在 Cursor 或 VS Code 可以调用 GLM 等大模型, 直接完成 Jupyter Notebook 中的全部代码填写. 因此, 如果最终提交的作业只有一份 “完整可运行的代码”, 而没有真实反映你的学习与实现过程, 作业分数会非常低.

若希望获得较高分数, 必须在实验操作过程中如实记录自己的思考、尝试与求助过程, 并整理为 Markdown 文件随作业一并提交. 记录内容至少应包括:

1. 你查阅了哪些资料.
2. 你向 AI 提出了哪些问题.
3. 你与 AI 互动的具体方式, 以及这些互动如何帮助你理解和完成实验.

Markdown 文件记录得越详细, 就越能证明该实验过程主要由你独立完成, 而不是直接依赖工具生成最终答案.

在初学阶段, 除选做题外, 本实验中存在的代码空缺不得直接使用 AI 工具自动补全, 你必须自己手写实现. 你可以向 AI 询问 “下一步应使用什么工具” 或 “某个 API 的作用是什么”, 但相关提问与回答过程同样需要如实记录在 Markdown 文件中, 作为学习过程的证明.

选做题不做要求, 你可以自由使用高级的 AI 工具.